

Project – Implementing Tutorials

CS 331 Spring 2026

1. Introduction

In your course project, you will spend at least 15 hours developing a software system of your choice. Time will be spent specifying requirements, testing, writing code, interacting with AI, etc.

2. Develop Software

Build a working software artifact that addresses a real-world problem. At minimum, this should be a prototype or proof-of-concept that demonstrates one or more core features. Focus on functionality over completeness. Projects with a small, working core are preferred over incomplete, overly ambitious systems.

Your development process should follow software engineering practices discussed in this course. This includes iteratively refining requirements, making design decisions, implementing features, and testing behavior. You are encouraged to start small (e.g., a minimal viable product) and incrementally expand that system as time permits.

You may use AI tools (e.g., code generation, debugging assistance, design support) in any capacity. However, all AI usage must be clearly documented as part of your engineering process. This includes what tools were used, how they were used, and how you validated or modified their outputs. Treat AI as a collaborator in the engineering process. AI may generate code. Just note that you must be able to explain the code and justify the decisions related to the software.

The final system should demonstrate:

- A clearly defined and motivated real-world problem being addressed by software
- At least one implemented core feature aligned with your requirements
- Evidence that you (not AI) completed the overall engineering process
- Basic validation through testing or manual verification

You won't be evaluated on building a production-ready system. You are being evaluated on whether you can take an idea, apply engineering discipline, and produce a working, defensible software artifact.

3. Track your Time

After every session you work on developing this project, take a minute to write down: (1) how much time you spent coding, (2) the tasks you worked on, and (3) what you accomplished. If you're spending lots of time dealing with bugs and not accomplishing much, that's ok – just document it.

4. Submission

Submit your code, paper, presentation materials (e.g., pptx), and any other supporting documents in **one zip** file. Only one submission if required per group. Please clearly indicate the group members in the submission.

If you use AI to generate any artifacts of your system (code, the paper, presentation slides, etc.), you must export and document your interactions with AI and include them in your project zip.

Any suspected use of AI that is not documented will be discussed on a case-by-case basis as a potential academic integrity violation. Please be honest with your AI use and don't risk a 0 on this project for dishonesty. AI use is fully permitted, **but it must not be used to misrepresent AI-generated work as your own.**

You will be graded according to how well you completed the engineering process. Honest use of AI will not negatively impact your grade.

5. Write a Paper, Present, and Discuss (via an Interview) your Software

The technical paper outlined below is worth 40 points. A presentation of your project will be worth 30 points. An individual interview on your implementation and engineering process will be worth 30 points. Your code will not be directly assessed but must also be submitted.

Paper

- I. Introduction (5 points)
 - a. Start with a short, high-level description of the system.
 - b. **Must-Have: Motivation** for why your software is worth building.
- II. Time Log (5 points)
 - a. Create a table with three columns “Time Spent”, “Tasks”, “Accomplishments”.
 - b. **Must-Have:** Short explanations (i.e., ~sentence) of **work sessions totaling 15+ hours**. For example, “2 hours: developed a testing procedure that is aligned with our core features.”
- III. Requirements (5 points)
 - a. **Must-Have:** informal descriptions of **core functional and non-functional requirements**, including status (e.g., complete, in-progress, not started) and explanation for functional ones.
 - b. Note: You may reuse and revise any content from your Requirements Discovery paper.
- IV. Design (5 points)
 - a. **Must-Haves:** Explanation of **frameworks** (e.g., Django, FastAPI, etc.) and how your code is structured. Also, description of one or more **design patterns** that are used in your system.
- V. Implementation (10 points)
 - a. Describe the system implementation in general.
 - b. **Must-Have:** Thorough explanations of **exemplary pieces of code** for each student. These exemplars must be substantial portions of code that address one or more functional requirements. Paste code snippet(s) into your paper and explain them.
- VI. Test Procedure (10 points)
 - a. Test the core features of your software using manual and/or automated tests. Ideally, all core functional requirements should be tested, and all lines of code should be covered by tests.
 - b. **Must-Have:** At least 5 *test procedures*, including 1+ manual and 1+ automated.
- VII. References (0 points – but points will be deducted for missing or incorrect references)
 - a. Cite the tutorial and any other resources you used.

Presentation

The presentation should cover your project requirements, design, implementation, and tests. The focus of your presentation should be the core features of the software (whether or not they are fully implemented) and a demonstration of your implementation. The presentation will be assessed as follows:

- System Overview of Requirements, Design, and Testing = 10 points
- Demonstration of Implementation and Functionalities = 10 points
- Quality of Presentation = 10 points

Presentation Length Requirements	Minimum Length (minutes)	Maximum Length (minutes)
Individual	5	10
Groups of Two	8	13
Groups of Three or More	10	15

Individual Interview

Students will schedule individual interviews (not in groups) with Dr. Gubanyi to discuss the software, focusing on the **Must-Haves** listed in the paper above. In particular, students should be prepared to discuss: (1) exemplary pieces of code in depth, (2) how the software works at a high level, and (3) the overall engineering process that you used to build this software. This will emulate real-world interviews for software engineers.